

Neura Companion — Ultimate Setup & Community Integration

This document merges all components, workflows, and community feedback systems into a single, cohesive guide and checklist.

1. Folder Structure & Modules

```
NeuraCompanion/  
├── packages/                # Refactored feature packages  
│   ├── core/  
│   ├── ritual_mode/  
│   ├── dream_visualizer/  
│   ├── mood_tracker/  
│   ├── error_handling/  
│   ├── voice_journaling/  
│   ├── mirror_mode/  
│   ├── store_front/  
│   ├── social_feed/  
│   ├── neuro_fetch/  
│   └── neuro_responsive/  
├── src/                    # App orchestrator  
│   ├── app.dart  
│   ├── main.dart  
│   └── pubspec.yaml  
├── assets/  
│   ├── neura_portrait.png  
│   ├── install_page.html  
│   ├── AppCast.xml  
│   └── store_assets/  
├── docs/  
│   ├── architecture.md  
│   ├── adding_module.md  
│   └── publish.md  
├── ci/  
│   ├── main.yml  
│   └── publish_packages.yml  
├── build_scripts/          # Automation scripts  
│   ├── generate_placeholders.sh  
│   ├── scaffold_modules.sh # Creates package scaffolds  
│   ├── run_tests.sh        # Runs unit & integration tests  
│   ├── build_all.sh         # Builds Flutter targets for all platforms  
│   ├── package_installers.sh # Packages builds into installers  
│   └── master_build.bat     # Master script to invoke all above
```

```

**Master Build Script (build_scripts/master_build.bat)
```batch
@echo off
REM -----
REM Master Build for Neura Companion
REM -----
REM 1. Generate asset placeholders
bash build_scripts/generate_placeholders.sh

REM 2. Scaffold feature packages
bash build_scripts/scaffold_modules.sh

REM 3. Run all tests
bash build_scripts/run_tests.sh
if %ERRORLEVEL% NEQ 0 (
 echo Tests failed. Aborting build.
 exit /b %ERRORLEVEL%
)

REM 4. Build all Flutter targets
bash build_scripts/build_all.sh

REM 5. Package installers
bash build_scripts/package_installers.sh

REM 6. Package individual feature zips
echo Packaging feature modules...
for %%p in (core ritual_mode dream_visualizer mood_tracker error_handling
voice_journaling mirror_mode store_front social_feed neuro_fetch
neuro_responsive) do (
 rd /s /q temp_package
 mkdir temp_package\%%p
 xcopy packages\%%p temp_package\%%p /E /I /Y
 powershell -Command "Compress-Archive -Path temp_package\%%p -DestinationPath
installers\%%p.zip -Force"
)
rd /s /q temp_package

echo Master build complete. Output: NeuraCompanion.zip and individual module
zips in installers\
pause

```

```

| |— generate_placeholders.sh
| |— scaffold_modules.sh # Creates package scaffolds
| |— run_tests.sh # Runs unit & integration tests

```

```

| ├── build_all.sh # Builds Flutter targets for all platforms
| ├── package_installers.sh # Packages builds into installers
| └── master_build.bat # Master script to invoke all above
├── installers/
| ├── Windows/
| ├── macOS/
| ├── Linux/
| ├── Android/
| ├── iOS/
| └── Web/
├── test/ # Global tests
├── integration_test/ # E2E tests
└── NeuraCompanion.zip # Final build

```

```

NeuraCompanion/ ├── packages/ # Refactored feature packages | ├── core/ | ├── ritual_mode/
| ├── dream_visualizer/ | ├── mood_tracker/ | ├── error_handling/ | ├── voice_journaling/ |
| ├── mirror_mode/ | ├── store_front/ | ├── social_feed/ | ├── neuro_fetch/ | └──
neuro_responsive/ ├── src/ # App orchestrator | ├── app.dart | ├── main.dart | └──
pubspec.yaml ├── assets/ | ├── neura_portrait.png | ├── install_page.html | ├── AppCast.xml
| ├── store_assets/ ├── docs/ | ├── architecture.md | ├── adding_module.md | └──
publish.md ├── ci/ | ├── main.yml | ├── publish_packages.yml ├── build_scripts/ | └──
setup.bat ├── installers/ | ├── Windows/ | ├── macOS/ | ├── Linux/ | ├── Android/ |
| ├── iOS/ | └── Web/ ├── test/ # Global tests ├── integration_test/ # E2E tests └──
NeuraCompanion.zip # Final build

```

```

2. Community & Feedback Integration

A. Feedback Widget
- **File**: `packages/social_feed/lib/ui/feedback_form.dart`
- **Description**: A form to collect user feedback with star ratings, comments,
and optional email.
- **Submit Logic**: Stores locally and optionally sends to Matomo or GitHub
Issue.

```dart
class FeedbackForm extends StatefulWidget { /* ... */ }

```

B. Feedback Service

• **File:** `packages/social_feed/lib/services/feedback_service.dart`

```

class FeedbackService {
  final Box _box = Hive.box('feedback');

```

```

Future<void> submit(String comment, int rating) async {
  final entry = {'comment': comment, 'rating': rating, 'time':
DateTime.now().toIso8601String()};
  await _box.add(entry);
  if (_matomoConfigured) {
    await MatomoService(_matomoUrl, _siteId).trackEvent('Feedback', 'submit');
  }
}
}
}

```

C. Upvotes & Community Board

- **Upvote Button:**

```

IconButton(icon: Icon(Icons.thumb_up), onPressed: () =>
UpvoteService().upvote(id));

```

- **Issue Linking:** Optional toggle to open a GitHub Issue via API.
- **Board UI:** `lib/ui/community_dashboard.dart` lists top-voted features and recent feedback.

```

ListView(children: feedbackList.map((f) => FeedbackTile(f)).toList());

```

3. Implementation Roadmap & Quality Checklist

Full Checklist Summary:

- Phase 1: Core Foundations
- Phase 2: Module Integration & Code Wiring
- Phase 3: Self-Sufficient Services & Hosting
- Phase 4: Release Quality & Automation
- Phase 5: Documentation & Developer Experience
- Phase 6: Community & Feedback Launch

Pre-Review: Before starting, verify that every module, graphic asset, animation, and workflow aligns with Neura's culture of empowerment, accessibility, and reliability.

Phase 0: Neura Animated Persona & Voice Integration *Tasks:* Mobile-ready animations, TTS/STT, personality engine with voice and animation options. - [] **Asset Integration:** - Place `A_digital_illustration_features_a_futuristic_fema.png` into `packages/core/assets/animations/neura_wave.png`. - Import `assets/neura_portrait.png`, `animations/neura_wave.png`, and all Rive/Lottie files into the same folder. - Update `pubspec.yaml` in core package:

```

flutter:
  assets:

```

- assets/neura_portrait.png
- assets/animations/neura_wave.png
- assets/animations/your_neura_intro.riv

- **Widget Reference:** In `packages/core/lib/ui/neura_widget.dart`, map states to assets and animations. - **Placeholder Generator Script:** Create `build_scripts/generate_placeholders.sh` to auto-generate empty files:

```
#!/usr/bin/env bash
BASE=packages/core/assets/animations
mkdir -p $BASE
touch packages/core/assets/neura_portrait.png
for mood in wave idle speaking listening happy excited sad thinking curious
welcoming encouraging motivated calm reflective confident empathetic; do
  if [[ $mood == "happy" || $mood == "sad" ]]; then
    touch $BASE/neura_${mood}.png
  else
    touch $BASE/your_neura_${mood}.riv
  fi
done
chmod +x build_scripts/generate_placeholders.sh
```

Run `bash build_scripts/generate_placeholders.sh` to scaffold all needed placeholders.

- Place `A_digital_illustration_features_a_futuristic,_fema.png` into `packages/core/assets/animations/neura_wave.png`.
- Import `assets/neura_portrait.png`, `animations/neura_wave.png`, and all Rive/Lottie files into the same folder.
- Update `pubspec.yaml` in core package:

```
flutter:
  assets:
    - assets/neura_portrait.png
    - assets/animations/neura_wave.png
    - assets/animations/your_neura_intro.riv
```

- **Widget Reference:** In `packages/core/lib/ui/neura_widget.dart`, map states to assets and animations:

```
Widget build(BuildContext context) {
  switch (_currentState) {
    case 'greeting':
      return Image.asset('assets/animations/neura_wave.png');
    case 'welcoming':
      return RiveAnimation.asset('assets/animations/
```

```

your_neura_welcome.riv');
    case 'speaking':
        return RiveAnimation.asset('assets/animations/
your_neura_speaking.riv');
    case 'listening':
        return RiveAnimation.asset('assets/animations/
your_neura_listening.riv');
    case 'happy':
        return Image.asset('assets/animations/neura_happy.png');
    case 'excited':
        return RiveAnimation.asset('assets/animations/
your_neura_excited.riv');
    case 'sad':
        return Image.asset('assets/animations/neura_sad.png');
    case 'thinking':
        return RiveAnimation.asset('assets/animations/
your_neura_thinking.riv');
    case 'curious':
        return RiveAnimation.asset('assets/animations/
your_neura_curious.riv');
    case 'encouraging':
        return RiveAnimation.asset('assets/animations/
your_neura_encouraging.riv');
    case 'motivated':
        return RiveAnimation.asset('assets/animations/
your_neura_motivated.riv');
    case 'calm':
        return RiveAnimation.asset('assets/animations/your_neura_calm.riv');
    case 'reflective':
        return RiveAnimation.asset('assets/animations/
your_neura_reflective.riv');
    case 'confident':
        return RiveAnimation.asset('assets/animations/
your_neura_confident.riv');
    case 'empathetic':
        return RiveAnimation.asset('assets/animations/
your_neura_empathetic.riv');
    default:
        return RiveAnimation.asset('assets/animations/your_neura_idle.riv');
}
}
}

```